FUNCTIONAL SPECIFICATION DOCUMENT

SignalForge

AI-Powered Trading Signal System

Complete functional specification covering all 12 system layers, data flow, API, database schema, and configuration

Version: 1.0 | **Classification:** Confidential

Date: 25 June 2026

Prepared by: Hermes Agent (AI Assistant)

Requested by: Fazrial

Status: Final

Document Control

Version	Date	Author	Description
1.0	25 Jun 2026	Hermes Agent	Initial FSD complete functional specification

Table of Contents

1. System Overview

SignalForge is a single-process asynchronous Python application that runs continuously on a Linux serv to Binance via WebSocket for live price data, processes market data through a 12-layer analysis pi confidence trading signals to the user via a Telegram bot.

1.1 Architectural Summary

```
                    SignalForge Pipeline

  Binance WS  Layer 0: Data Ingestion

                  Layer 1: Feature Engineering

                  Layer 2: Pattern & Structure Detection

                  Layer 3: Confluence Scoring

                  Layer 4: MTF Bias Engine

                  Layer 5: Cooldown & Dedup Gate

                  Layer 6: LLM Reasoning Engine

                  Layer 7: Filter Gate (10 filters)
```

```
   Layer 8: Risk & Sizing Engine

   Layer 9: Execution & Delivery (Telegram)

Layer 10: Logging & Tracking
         SQLite Database

Layer 11: Feedback & Optimization
     Weekly Report / Prompt Tuning

Layer 12: Health Monitoring
   Watchdog / 8080 Health Endpoint
```

1.2 Key Technical Decisions

Decision	Rationale
Single asyncio process	Simpler deployment, no container orchestration needed, lower AWS co
SQLite instead of PostgreSQL	Single-user system; no concurrent write contention; zero maintenance
LLM via 9router (hermes-main)	Routes to Claude Opus 4.6 cheaper than direct OpenAI API, better reas
Telegram bot (not web UI)	Instant delivery, mobile-friendly, zero frontend development
Paper trading built-in	Test strategies with $10K virtual balance before going live
systemd supervision	Zero-dependency auto-restart on crash; built-in logging rotation

1.3 Runtime Environment

Parameter	Value
Process type	Single async Python 3.11+ process
Event loop	asyncio
Supervision	systemd (restart always, 5s delay)
Memory baseline	~480 MB peak
CPU usage	<5% idle, spikes to 20% during LLM calls
Uptime target	99.9% (only down for deploys)

---

2. Module Specifications

2.1 Layer 0 Data Ingestion

Files: `data/websocket_feed.py` , `data/fetcher.py` , `data/multi_asset_feed.py`

Purpose: Establish and maintain live data streams from Binance and fetch historical OHLCV data.

WebSocket Feed (`WebSocketFeed`): - Connects to Binance WebSocket Stream API for 1-minute kline symbol streams concurrently: `btcusdt@kline_1m`, `ethusdt@kline_1m`, etc. - Auto-reconnect on disconnect (1s 5s 30s cap) - Each tick updates a shared `latest_prices` dict with symbol price mapping - Fires an new candle close

OHLCV Fetcher (`DataFetcher`): - Uses ccxt to fetch historical OHLCV data from Binance REST AP timeframes: `1h`, `4h`, `1d` - Configurable lookback: 300 bars per timeframe by default - Implemer requests per second) - Fallback to cached data on API failure

Multi-Asset Feed (`MultiAssetFeed`): - Orchestrates WebSocket connections for all enabled assets - Mar (start, stop, health check) - Tracks per-symbol tick counts and last tick timestamps

Configuration:

```
# config/assets.py
ASSETS = [
    AssetConfig(symbol="BTC/USDT", binance_symbol="BTCUSDT", timeframes=["1h","4h","1d"], primary_t
    AssetConfig(symbol="ETH/USDT", binance_symbol="ETHUSDT", timeframes=["1h","4h","1d"], primary_t
    AssetConfig(symbol="BNB/USDT", binance_symbol="BNBUSDT", timeframes=["1h","4h","1d"], primary_t
    AssetConfig(symbol="SOL/USDT", binance_symbol="SOLUSDT", timeframes=["1h","4h","1d"], primary_t
]
```

Edge Cases Handled: - WebSocket disconnect during idle periods auto-reconnect within 30s - Late can accept up to 5s late - Empty response from ccxt REST retry with exponential backoff, fall back to cache

---

2.2 Layer 1 Feature Engineering

Files: `signals/pipeline.py`

Purpose: Transform raw OHLCV data into calculated technical indicators and structural features.

Indicators Calculated (via pandas-ta): - **Momentum:** RSI(14), Stochastic(14,3,3), CCI(20), Williams EMA(20/50/200), VWAP, Supertrend(10,3), Ichimoku Cloud, ADX(14) - **Volatility:** ATR(14), Bollinger Band **Volume:** OBV, RVOL (relative volume), Volume Profile POC - **MACD:** 12, 26, 9 line, signal, histogram, hist

Structural Features: - Swing points (pivot highs/lows with configurable lookback) - Market structure s ranging based on HH/HL/LH/LL) - S/R levels (swing point clustering within 0.3% tolerance) - Premium/Disc range midpoint)

Candlestick Pattern Detection: - Single candle: Hammer, Shooting Star, Doji (Dragonfly, Gravestone Bullish/Bearish Engulfing, Piercing Line, Dark Cloud Cover - Three candle: Morning Star, Evening Star, Th Black Crows

Update Frequency: On each new 1m candle close from WebSocket, but full re-analysis only every 60 s cycle).

2.3 Layer 2 Pattern & Structure Detection

Files: `signals/swing_detector.py`, `signals/fvg_detector.py`, `signals/srm_runner.py`

Purpose: Identify Smart Money Concepts (SMC/ICT) patterns and classic chart patterns.

Swing Detector (`SwingDetector`): - Identifies swing highs and lows using pivot point analysis with Minimum swing height threshold to filter noise (0.3% of price) - Tracks swing history for trend structure (H

Break of Structure (BOS): - Bullish BOS: price breaks above previous swing high in an uptrend - B below previous swing low in a downtrend - Requires 1 candle close beyond the swing point for confirmatio

Change of Structure (ChOS): - Bullish ChOS: in a downtrend, price breaks above the most recent low an uptrend, price breaks below the most recent higher low - Priority over BOS signals potential trend reve

Fair Value Gap (FVG) Detection: - 3-candle imbalance: Candle 3 high/low does not overlap Candle 1 size: 0.1% of price to filter noise - Tracks FVG age (when formed, how many candles since) - Identifies FVG (mitigation) - Supports both bullish FVG (gap up) and bearish FVG (gap down)

Order Block (OB) Detection: - Last opposing candle before an impulsive move (body > 1.5× ATR) candle before bullish impulse - Bearish OB: last bullish candle before bearish impulse - OB zone = candle

Liquidity Grab Detection: - Price wicks beyond a recent swing high/low - Candle closes back inside th volume on the sweep candle - Reversal confirmation on the following candle

Chart Pattern Detection: - Head & Shoulders / Inverse H&S - Double Top / Double Bottom - A Symmetrical Triangles - Bull / Bear Flags - Wedges (Rising / Falling) - Cup & Handle - TTM Squeeze (Bolli Channel)

Divergence Detection: - RSI regular divergence (price HH + RSI LH = bearish divergence) - MACD reg + MACD LH) - Hidden divergence (pullbacks in trends) - Force index divergence

---

2.4 Layer 3 Confluence Scoring Engine

Files: `signals/confluence.py`, `signals/mtf_bias.py`

Purpose: Aggregate all detected signals into a weighted confluence score.

Signal Classification:

Tier	Weight	Signals
Tier 1 Structure	3× per signal	BOS/ChOS, Market Structure, S/R Break/Hold, OB Reaction, MTF Bias Alignment
Tier 2 Trigger	2× per signal	FVG Entry, Liquidity Grab + Reversal, Impulsive Candle at Key Level, Candlesti Squeeze Breakout, Volume Surge, Chart Pattern Breakout
Tier 3 Confirmation	1× per signal	RSI Alignment (<65/ >35), MACD Histogram direction, EMA Alignment, VWAP p Chain Flow, Sentiment, Funding Rate, Killzone Timing, Correlation

Scoring Formula:

```
Total Score = (Tier 1 signals × 3) + (Tier 2 signals × 2) + (Tier 3 signals × 1)
```

Thresholds:

Score Range	Classification	Action
07	SUPPRESSED	Log silently. No LLM call. No signal delivered.

Score Range	Classification	Action
810	STANDARD	Trigger LLM. Normal position size if confidence 75%.
1114	STRONG	Trigger LLM. Priority delivery.
15+	PREMIUM	Trigger LLM. Max allowable position size.

MTF Bias Engine: - Calculates directional bias for 1D, 4H, and 1H timeframes - If all three do not a suppressed until alignment - Bias determined by: EMA alignment (20/50/200), HH/HL structure, ADX dire 4H candle close and on-demand during SMC analysis cycle

---

2.5 Layer 4 MTF Bias Engine

See Layer 3 MTF Bias Engine section above for full specification.

2.6 Layer 5 Cooldown & Dedup Gate

Files: `db/schema.py` (cooldowns table)

Purpose: Prevent signal spam, revenge trading, and duplicate signals.

Cooldown Rules:

Condition	Cooldown Duration
Normal signal delivered	30 minutes
Stop loss hit	2 hours (revenge trade prevention)
Consecutive losses (3+)	4 hours
Daily loss limit hit (4%)	Until next trading day

Implementation: - Cooldowns stored in SQLite `cooldowns` table with asset symbol and expiry timestam 6 (LLM) to avoid unnecessary LLM costs - Dedup: same direction signal for same asset within cooldown su

---

2.7 Layer 6 LLM Reasoning Engine

Files: `signals/llm_engine.py`, `signals/prompt_builder.py`

Purpose: Use a Large Language Model to evaluate the full market context and decide whether to issue a

Model Configuration:

```
OPENAI_API_KEY=sk-***           # 9router API key
OPENAI_BASE_URL=http://localhost:20128/v1
OPENAI_MODEL=hermes-main         # Routes to Claude Opus 4.6 via 9router
OPENAI_FALLBACK=hermes-main      # Same model for fallback (9router handles failover internally)
```

Prompt Structure: The LLM prompt (built by `prompt_builder.py`) includes: 1. System instruction defi expert SMC/ICT trader 2. Current market context: price, timeframe bias (1D/4H/1H), support/resistance l

ICT features: BOS, ChOS, FVGs, OBs, liquidity grabs 4. Technical indicators: RSI, MACD, EMA alignment Candlestick patterns detected 6. External data: Fear & Greed Index, news sentiment (if available), cor positions and account context 8. Output format specification (JSON with strict schema)

LLM Response Schema:

```
{
  "signal": "LONG" | "SHORT" | "PASS",
  "confidence": 0-100,
  "entry_price": float,
  "tp1/tp2/tp3": float,
  "stop_loss": float,
  "reasoning": "string",
  "invalidation": "string"
}
```

Retry Logic: - Max retries: 3 (LLM_MAX_RETRIES) - Backoff: 1s linear between retries - Fallback: Uses OP last attempt - Timeout: 60s per request (LLM_TIMEOUT) - On total failure: returns PASS signal with "API er

---

2.8 Layer 7 Filter Gate

Files: `signals/pipeline.py` (filter gate logic in SMC analysis loop)

Purpose: Apply 10 independent filters that ALL must pass for signal delivery.

#	Filter	Condition	Implementation
1	Confidence	LLM confidence 75%	Compare `signal.conf`
2	Risk/Reward	R:R ratio 1.5	Compare entry to SL/TP
3	MTF Alignment	Daily + 4H + 1H agree	Bias engine output chec
4	Cooldown	No signal for this asset in 3060 min	SQLite cooldowns table
5	Active Signals Cap	Open positions < 3	Position tracker count
6	Portfolio Heat	Total open risk < 6% of account	Sum of all open position
7	News Buffer	No high-impact event within ±2h	Economic calendar chec
8	Volatility Regime	ATR not in extreme (>3σ above avg)	ATR stats comparison
9	Sentiment Extreme	Fear & Greed between 10 and 90	F&G index check
10	Spread	Bid-ask spread < 0.1% of price	Orderbook check (plann

---

2.9 Layer 8 Risk & Sizing Engine

Files: `trading/paper_trade.py`

Purpose: Calculate dynamic position sizes based on LLM confidence and account risk parameters.

Formula:

```
Position Size = (Account Balance × Risk%) ÷ |Entry Price  Stop Loss Price|
```

Risk Tiers by Confidence:

Confidence	Account Risk %
7580%	1.0%
8090%	1.5%
90%+	2.0%

Hard Rules (System-Enforced): - Max risk per trade: 2% of account - Max concurrent positions: 3 - Stop loss mandatory on every signal - Daily loss limit: 4% (auto-pause) - Weekly loss limit: 8% (reduce ma

Take Profit Strategy:

Level	Close %	Action
TP1	40%	Move SL to breakeven
TP2	30%	Trail SL below last HL (longs)
TP3	30% (remainder)	Position closed

2.10 Layer 9 Execution & Delivery

Files: `delivery/telegram_bot.py` , `delivery/telegram_commands.py`

Purpose: Format and deliver signals to the user via Telegram bot.

Telegram Bot (`TelegramBot`): - Uses direct HTTP API calls (no python-telegram-bot library required `parse_mode=HTML` for formatted output - Emoji indicators: LONG / SHORT - Supports `/` commands via lon

Telegram Commands (`TelegramCommandHandler`):

Command	Description	Response
`/status` or `/positions`	Show open positions	List of active papers with en
`/stats`	Show 7-day performance	Win rate, avg R:R, profit fact
`/history [days]`	Show recent closed positions	Last N closed trades with P&
`/health`	Show system component health	pipeline, llm, database, web
`/close <id> <price>`	Manually close a position	Confirmation with P&L

Signal Message Format:

```
 LONG BTC/USDT
Entry: $67,420
TP1: $68,900 (+2.2%)  40%
```

```
TP2: $69,800 (+3.5%)  30%
TP3: $71,200 (+5.6%)  30%
SL: $66,100 (-2.0%)
R:R 1:2.8 | Confidence: 84% | Risk: 1.5%

Reasoning: [LLM reasoning text]
```

Bot Configuration: - Token: from `TELEGRAM_BOT_TOKEN` env var - Chat ID: from `TELEGRAM_CHAT_I` `@tradingforgee_bot`

2.11 Layer 10 Logging & Tracking

Files: `db/schema.py`, `signals/position_tracker.py`, `signals/win_rate.py`

Purpose: Log every signal and trade outcome with full context for audit and performance analysis.

Database: SQLite at `db/signalforge.db`

All signals (delivered and suppressed) are logged with: - Timestamp, symbol, direction, entry price - Cor (Tier 1/2/3 count) - LLM confidence, model used, latency - Filter gate results (which filters passed/f (delivered/suppressed/pending)

Trade tracking: - Auto-detection of TP/SL hits via paper engine - Every closed trade logged with P&L classification - Average R:R realized vs expected

---

2.12 Layer 11 Feedback & Optimization

Files: `monitoring/weekly_report.py`, `scripts/send_weekly_report.py`

Purpose: Generate weekly performance reports and feed results back into system improvement.

Weekly Report (every Monday 07:00 WIB): - Win rate (7-day rolling) - Number of trades and signals vs realized) - Profit factor - Best/worst setup analysis - System uptime

Cron Schedule: - Runs every Sunday 00:00 UTC via Hermes cron - No-agent mode (0 token co `send_weekly_report.py`

2.13 Layer 12 Health Monitoring

Files: `monitoring/health_endpoint.py`, `monitoring/watchdog.py`, `monitoring/error_alerter.py`

Purpose: Ensure system reliability with automatic failure detection and alerting.

Health HTTP Endpoint (port 8080):

```
GET /health  {"status": "ok", "uptime_seconds": N, "components": {...}}
GET /health/ws  {"symbols": {"BTC/USDT": {"connected": true, "last_tick_age_s": N, "total_ticks": N
```

Component Health Tracking:

Component	Status Values	Details
pipeline	ok / degraded / down	Cycle completion, assets analyzed

Component	Status Values	Details
llm	ok / degraded / down	Model used, latest response time
database	ok / degraded / down	Connection status
websocket	ok / degraded / down	Symbols connected, live status

Health Watchdog: - Runs every 5 minutes - Checks: WebSocket connection, LLM API responsiveness, space, RAM usage - Triggers Telegram alert on any failure - Systemd auto-restart on crash (RestartSec=5)

Error Alerter: - Monitors log file for ERROR level messages - Sends Telegram alert on critical errors (pipe disconnect) - Runs every 60 seconds (configurable) - Flood control: same error deduped within 5 minutes

---

3. Data Flow Architecture

3.1 Main Loop (60-second cycle)

```
1. Fetch OHLCV for all 4 assets (1H, 4H, 1D)
2. For each asset (3s delay between assets):
   a. Calculate indicators (RSI, MACD, EMA, ATR, BB, etc.)
   b. Detect SMC structures (BOS, ChOS, FVG, OB, liquidity grab)
   c. Score confluence (Tier 1 + Tier 2 + Tier 3)
   d. If score >= 8:
      - Check cooldown
      - Build LLM prompt with full context
      - Call LLM (hermes-main  Claude Opus 4.6)
      - Parse LLM response
      - Apply filter gate (10 filters)
      - If all pass: calculate position size, deliver signal via Telegram
3. Paper engine auto-tick: check open positions for TP/SL hits
4. Update health status
5. Sleep until next 60s cycle
```

3.2 WebSocket Data Flow (Continuous)

```
Binance WS Stream  on_tick callback  update latest_prices dict
                                     update per-symbol tick count
                                     update WS health status
```

3.3 Command Flow (On-Demand)

```
User sends /status  getUpdates polling  TelegramCommandHandler
                                        query PositionTracker
                                        format HTML response
                                        send via TelegramBot.send()
```

---

4. API Specifications

4.1 Internal Health API

Endpoint: `GET /health`

Response:

```
{
  "status": "ok",
  "uptime_seconds": 3600.5,
  "components": {
    "pipeline": {"status": "ok", "details": "Cycle complete, 4 assets analyzed"},
    "llm": {"status": "ok", "details": "Model: hermes-main"},
    "database": {"status": "ok", "details": ""},
    "websocket": {"status": "ok", "details": ""}
  },
  "version": "1.0"
}
```

Endpoint: `GET /health/ws`

Response:

```
{
  "symbols": {
    "BTC/USDT": {"connected": true, "last_tick_age_s": 3.5, "total_ticks": 1520},
    "ETH/USDT": {"connected": true, "last_tick_age_s": 3.5, "total_ticks": 1520}
  },
  "total_symbols": 4
}
```

4.2 Telegram Bot Commands

Command	HTTP Method	Parameters	Response
`/status`	getUpdates handle	None	HTML list of op
`/stats`	getUpdates handle	None	HTML perform
`/history [days]`	getUpdates handle	days (int, default 7)	HTML closed p
`/health`	getUpdates handle	None	HTML compon
`/close <id> <price>`	getUpdates handle	id (int), price (float)	HTML confirma

Telegram API called internally:

```
POST https://api.telegram.org/bot<TOKEN>/sendMessage
  chat_id: <CHAT_ID>
  text: <HTML formatted message>
  parse_mode: HTML
```

5. Database Schema

File: `db/signalforge.db`

5.1 Entity Relationship Diagram (Textual)

```
candles (1)  signals ()  paper_trades ()  paper_balance (1)

                                    positions ()

signals ()  cooldowns (1) [per symbol]
```

5.2 Table Definitions

`candles` table:

```
CREATE TABLE candles (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol TEXT NOT NULL,
    timeframe TEXT NOT NULL,
    timestamp INTEGER NOT NULL,
    open REAL, high REAL, low REAL, close REAL,
    volume REAL,
    UNIQUE(symbol, timeframe, timestamp)
);
```

`signals` table:

```
CREATE TABLE signals (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp INTEGER NOT NULL,
    symbol TEXT NOT NULL,
    direction TEXT,
    entry_price REAL,
    confidence INTEGER,
    rr_ratio REAL,
    confluence_score INTEGER,
    model_used TEXT,
    filter_results TEXT,
    status TEXT DEFAULT 'pending',
    delivered INTEGER DEFAULT 0,
    reasoning TEXT,
    error TEXT
);
```

`positions` table:

```
CREATE TABLE positions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    signal_id INTEGER,
    symbol TEXT NOT NULL,
    direction TEXT,
    entry_price REAL,
```

```
    stop_loss REAL,
    tp1 REAL, tp2 REAL, tp3 REAL,
    position_size REAL,
    status TEXT DEFAULT 'OPEN',
    opened_at INTEGER,
    closed_at INTEGER,
    pnl_usd REAL,
    pnl_pct REAL,
    FOREIGN KEY (signal_id) REFERENCES signals(id)
);
```

`paper_trades` **table:**

```
CREATE TABLE paper_trades (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    symbol TEXT NOT NULL,
    direction TEXT,
    entry_price REAL,
    sl REAL, tp1 REAL, tp2 REAL, tp3 REAL,
    position_size REAL,
    signal_id INTEGER,
    confidence INTEGER,
    status TEXT DEFAULT 'OPEN',
    open_price REAL,
    close_price REAL,
    pnl_usd REAL, pnl_pct REAL,
    rr_realized REAL,
    tp1_hit INTEGER DEFAULT 0,
    tp2_hit INTEGER DEFAULT 0,
    sl_hit INTEGER DEFAULT 0,
    opened_at INTEGER,
    closed_at INTEGER
);
```

`paper_balance` **table:**

```
CREATE TABLE paper_balance (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    balance REAL NOT NULL DEFAULT 10000.0,
    updated_at INTEGER NOT NULL
);
```

`cooldowns` **table:**

```
CREATE TABLE cooldowns (
    symbol TEXT PRIMARY KEY,
    expires_at INTEGER NOT NULL
);
```

`health_log` **table:**

```
CREATE TABLE health_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp INTEGER NOT NULL,
```

```
    component TEXT NOT NULL,
    status TEXT NOT NULL,
    details TEXT
);
```

`build_log` table:

```
CREATE TABLE build_log (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    sprint INTEGER,
    timestamp INTEGER NOT NULL,
    event TEXT NOT NULL,
    details TEXT
);
```

---

6. Error Handling & Edge Cases

6.1 Error Recovery Strategy

Error Type	Detection	Recovery
WebSocket disconnect	No tick for 60s	Auto-reconnect (exponential backof
LLM API timeout	60s timeout on request	Retry up to 3 times, then PASS
LLM JSON parse failure	Invalid JSON response	Retry (LLM regenerates), then PASS
Binance REST rate limit	HTTP 429 from ccxt	Exponential backoff, use cached dat
SQLite write failure	sqlite3.OperationalError	Retry after 1s; if persistent alert
system crash	Process exits	systemd auto-restart (RestartSec=5
Memory leak	Memory > 1GB threshold	Health watchdog triggers restart
Disk full	Write failure	Alert via Telegram, graceful shutdow

6.2 Edge Cases

Edge Case	Handling
No trades yet (new deployment)	`/status` returns "No open positions"
All LLM calls fail	Returns PASS, logged with error detail
Cooldown expires during LLM call	Checked at delivery, not at trigger
Multiple signals same minute	Dedup by symbol + direction
Empty WebSocket feed on startup	Graceful degradation: uses REST data until WS connects
SOL/USDT volatile spikes	Higher min_confluence_score (9 vs 8)

Edge Case	Handling
4 assets exceed processing window	3s stagger between assets keeps cycle under 60s

7. Performance Requirements

Metric	Requirement	Actual (Measured)
SMC analysis cycle time	<60s for 4 assets	~20-25s (with LLM ca
WebSocket tick latency	<500ms from exchange	~100-300ms
LLM response time	<30s per call	~15-20s (Claude Opus
REST OHLCV fetch	<5s per asset	~1-2s
Telegram delivery	<2s from decision	~500ms
Memory footprint	<1GB RSS	~480-530 MB peak
CPU usage (idle)	<10%	~3-5%
CPU usage (active)	<50%	~15-25%
Database size (30 days)	<100MB	~2MB currently
Concurrent WS streams	10+	4 currently, scalable t

8. Configuration Reference

8.1 Environment Variables (.env)

```
# === Telegram ===
TELEGRAM_BOT_TOKEN=***          # Bot token from @BotFather
TELEGRAM_CHAT_ID=955169177      # Your Telegram chat ID

# === LLM (9router) ===
OPENAI_API_KEY=sk-***           # 9router API key
OPENAI_BASE_URL=http://localhost:20128/v1
OPENAI_MODEL=hermes-main        # Routes to Claude Opus 4.6
OPENAI_FALLBACK=hermes-main     # Same model for fallback

# === Trading ===
PAPER_TRADING=true              # true = paper mode, false = live signals
```

8.2 Asset Configuration (config/assets.py)

See Section 2.1 for `AssetConfig` dataclass and `ASSETS` list. Key parameters per asset: - `symbol`: Trading `binance_symbol`: Binance format (e.g., "BTCUSDT") - `timeframes`: List of timeframes ["1h", "4h", "1 timeframe (usually "1h") - `min_confluence_score`: Threshold per asset (default 8, SOL=9) - `min_rr`: Min 1.5) - `enabled`: True/False to include in pipeline

9. Approval

Role	Name	Date
Product Owner	Fazrial	___/___/2026
Developer	Hermes Agent	25/06/2026
Reviewer		___/___/2026

End of Document SignalForge FSD v1.0 Confidential Do not distribute without authorization.